

Student Meal Planner

Semester Reflection

A Personal Leadership Document for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

Looking Back at the Semester

[NOTE: This is a draft based on what I know about your project. You'll want to personalize this heavily. Add your own feelings, specific moments, and details I wouldn't know about. The structure and voice are right, but the personal details need to be yours.]

This semester was the first time I built something from scratch that wasn't a tutorial project. Grubs4Scrubs started as an idea for a student meal planning app and turned into a full-stack project with a React frontend, an ASP.NET Core API, and a SQL Server database. Looking back, the technical skills were only part of what I learned. A bigger part was learning how to make decisions when there's no right answer in the textbook.

What I Learned Technically

Going into the semester, I knew some HTML and CSS. React was new. C# was new. SQL was something I'd done in a database course but never connected to a real application. By the end, I'd built a multi-page React app with routing, state management, and a custom CSS system. On the backend, I'd set up a layered architecture with dependency injection, written raw SQL queries in ADO.NET repositories, designed database tables for recipes, users, and shopping items, and built a full JWT authentication system with BCrypt password hashing. The auth system was the last major feature I built, and it pulled together everything: the domain model (User, DTOs), the repository (GetByEmail), the controller (register and login logic), and the middleware (token validation in Program.cs).

The thing that surprised me most was how much time goes into decisions that aren't about code. Choosing between Tailwind and custom CSS. Deciding whether to use folders or separate projects for the backend layers. Figuring out the right database connection string. These choices don't produce visible features, but getting them wrong costs hours of rework. I learned to spend time thinking before coding, which is something I definitely didn't do at the start.

What I Struggled With

The no-EF-Core constraint was the biggest challenge. Every tutorial and Stack Overflow answer assumes you're using Entity Framework. Finding examples of the repository pattern with raw ADO.NET took way more searching. I had to adapt EF Core examples to SqlConnection and SqlCommand, which meant understanding both approaches well enough to translate between them.

The connection string saga was frustrating. Switching from LocalDB to full SQL Server, getting the server name right, making sure the database existed in the right instance. It's the kind of problem that isn't hard conceptually but eats hours because the error messages aren't helpful. "Error Locating Server/Instance Specified" doesn't tell you which part of your connection string is wrong.

[NOTE: Add any personal struggles here. Time management, motivation dips, balancing with other courses, group project conflicts, whatever was real for you.]

What I'd Do Differently

I'd make architectural decisions earlier. The Tailwind mess happened because I started building pages without deciding on a CSS approach first. The flat project structure existed for weeks before I restructured it into layers. In both cases, the fix wasn't hard, but doing it after the fact meant touching every file instead of getting it right from the start.

I'd also write documentation as I go instead of at the end. The research docs I wrote close to when I did the work were detailed and specific. The ones I wrote later relied on memory and were harder to write. Even quick notes during development ("chose X because Y") would've made the formal docs much easier.

[NOTE: Add anything else you'd personally change. Study habits, asking for help sooner, starting the portfolio earlier, etc.]

How I Grew

At the start of the semester I couldn't read a React component and understand what it did. Now I can build one from an HTML mockup, wire it to a backend API, and explain the architectural decisions behind it. That's a concrete skill difference.

But the less obvious growth was in how I approach problems. I used to dive straight into code and figure it out as I went. Now I plan first. Not always perfectly, and sometimes I still jump in too early, but the instinct to think before building is new. The layered architecture research doc, the CSS decision research, the database design decisions. Those exist because I forced myself to reason through the choice before committing to it.

[NOTE: Add anything personal here about how you grew as a person, not just a developer. Confidence, communication with your coach, independence, anything that feels real.]

Goals for Next Semester

[NOTE: Fill in with your actual goals. Here are some prompts:]

- What technical skills do you want to learn next?
- What would you do differently in your project workflow?
- Are there soft skills (presentations, teamwork, time management) you want to improve?
- What part of this semester's work are you most proud of, and how do you build on it?